

**TITLE**

**GIT BRANCHING AND MERGE CONFLICT RESOLUTION IN DISTRIBUTED  
VERSION CONTROL SYSTEMS**

**STUDENT DETAILS**

**NAME - 1: RAGHAVA M [ENROLLMENT: 132049]  
NAME - 2: [GROUP MEMBER 2] [ENROLLMENT: 2]  
NAME - 3: [GROUP MEMBER 3] [ENROLLMENT: 3]**

**DEPARTMENT**

**COMPUTER ENGINEERING**

**ACADEMIC YEAR**

**2026-2027**

**SEMESTER**

**5<sup>TH</sup>**

**SUBJECT**

**OPEN SOURCE TECHNOLOGIES (01CE0526)**

**CLASS**

**5CE**

**BATCH**

**Batch 1**

---

RAJKOT-MORBI ROAD, AT & PO: GAURIDAD, RAJKOT - 360 003, GUJARAT, INDIA  
Ph: +91-281-2923112, +91-281-2924154-56 | email: [info@marwadiuniversity.ac.in](mailto:info@marwadiuniversity.ac.in) | [www.marwadiuniversity.ac.in](http://www.marwadiuniversity.ac.in)

## RUBRICS FOR COMPLEX ENGINEERING PROBLEM

| Sr. No.      | Assessment Criteria                                                    | Excellent                                                                                                                                                                               | Good                                                                                        | Satisfactory                                                                         | Poor                                                                                 | Marks     |
|--------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------|
| 1            | <b>Problem Analysis &amp; Open Source Technology Selection</b>         | <b>3 Marks:</b><br>Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.       | <b>2 Marks:</b><br>Good problem analysis with suitable technology selection and minor gaps. | <b>1 Mark:</b><br>Basic analysis with limited justification of technology selection. | <b>0 Marks:</b><br>Inadequate problem analysis and inappropriate selection.          | 3         |
| 2            | <b>System Design, Implementation Strategy &amp; Technical Analysis</b> | <b>4 Marks:</b> Well-defined system architecture, appropriate technology stack, version control, and comprehensive technical analysis including scalability, reliability, and security. | <b>3 Marks:</b><br>Good system design and technical discussion with minor omissions.        | <b>1-2 Marks:</b><br>Basic architecture and limited technical analysis.              | <b>0 Marks:</b><br>Poor or missing system design and technical analysis.             | 4         |
| 3            | <b>Testing, Documentation &amp; References</b>                         | <b>3 Marks:</b><br>Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.                                                 | <b>2 Marks:</b><br>Good documentation with minor formatting or testing gaps.                | <b>1 Mark:</b><br>Basic report with limited testing or references.                   | <b>0 Marks:</b><br>Incomplete report with poor documentation and missing references. | 3         |
| <b>Total</b> |                                                                        |                                                                                                                                                                                         |                                                                                             |                                                                                      |                                                                                      | <b>10</b> |

| Sr. No. | Complex Engineering Problem Activity / Problem Title | COs      | Date       | R1(4) | R2(3) | R3(4) | Marks (10) | Signature |
|---------|------------------------------------------------------|----------|------------|-------|-------|-------|------------|-----------|
| 1       | Git Branching and Merge Conflict Resolution          | CO2, CO4 | 05-10-2026 |       |       |       |            |           |

### Notes / Instructions

1. CEP activity must be completed in a group of maximum 3 students.
2. Students must strictly follow the given deadlines for submission.
3. The report must be prepared in the prescribed format and submitted as a DOCX file on Google Classroom.
4. A hard copy of the report must be submitted as discussed.
5. The report should include problem analysis, design/architecture, implementation, results, and challenges faced.
6. All group members must contribute equally and be prepared for evaluation/presentation.
7. Plagiarism or copying from other groups will not be accepted.
8. Each group member must understand the complete solution and should be able to explain their contribution during evaluation.

**Ms. Priyanka Mangi**

Subject Faculty

---

**TITLE: GIT BRANCHING AND MERGE CONFLICT RESOLUTION IN DISTRIBUTED VERSION CONTROL SYSTEMS**

**Course Name:** OPEN SOURCE TECHNOLOGIES (01CE0526)

**CO's Mapped:** CO2, CO4

**SDG's Mapped:** SDG 4 (Quality Education), SDG 9 (Industry, Innovation, and Infrastructure)

**1. Aim**

To analyse, design, and develop an engineering solution for the given complex engineering problem using appropriate knowledge and tools.

**2. Problem Statement**

In modern collaborative software engineering, distributed teams concurrently develop features across isolated Git branches. When multiple developers perform divergent modifications on identical lines of code within shared source files, automated merge algorithms fail, precipitating merge conflicts. The objective is to analyze the mechanics of merge conflicts, simulate a concurrent modification scenario between main and feature branches, perform manual conflict resolution through syntactic and semantic reconciliation, and synchronize the resolved state to a remote GitHub repository.

**3. Problem Analysis**

**a) What is the engineering problem?**

The engineering problem involves managing non-linear source code divergence in distributed version control systems where concurrent edits to identical lines prevent automated 3-way reconciliation, creating build breakage and workflow blockages unless systematically analyzed and resolved.

**b) Why is this problem complex?**

- ☒ Requires multiple engineering concepts (DAG graphs, tree hashing, 3-way merge algorithms).
- ☒ Requires analysis and decision making (evaluating competing line changes, preserving semantic correctness).
- ☒ Involves design constraints (maintaining linear commit integrity, adhering to branching rules).
- ☒ Requires use of engineering tools (Git CLI, GitHub remote cloud hosting, diff tools).
- ☒ Requires performance/security/safety consideration (preventing source regression and data overwrite).

**4. Knowledge and Tools Required**

**Engineering concepts used:**

1. Distributed Version Control Systems (DVCS) and Directed Acyclic Graph (DAG) commit trees.
2. Git 3-Way Merge Algorithm (common base ancestor, current HEAD branch, incoming branch).
3. Conflict Marker Demarcation Syntax (<<<<<< HEAD, =====, >>>>>> feature).
4. Git State Transitions (Working Directory → Staging Index → Local Repository → Remote Repository).

### Tools/Software/Hardware used:

1. Git CLI (Version 2.55.0 on Windows)
2. GitHub Remote Repository (<https://github.com/Raghavendra-2007/Git-Branching-and-Merge-Conflict-Resolution>)
3. Windows PowerShell / Visual Studio Code

### 5. Proposed Solution Design

#### Block Diagram / Architecture Diagram:

```
LOCAL REPOSITORY:
[Initial Commit: aee5835] (main)
    |
    +-----+-----+
    |               |
    v               v
(main)         (feature)
[a1a2f4b]     [452eb12]
    |               |
    +-----+-----+
    |               |
    v               v
[git merge] ---> CONFLICT DETECTED (index.html)
                  (<<<<<<< HEAD ... ===== ... >>>>>>> feature)
                  |
                  v
[Manual Resolution]
                  |
                  v
[Merge Commit: 971f733] ("Welcome to Main Website - Feature Update")
                  |
                  v (git push origin main, git push origin feature)
GITHUB REMOTE REPOSITORY:
https://github.com/Raghavendra-2007/Git-Branching-and-Merge-Conflict-Resolution
```

#### Brief Working:

The solution initializes a Git repository with main as default and creates a baseline index.html commit. An isolated feature branch is spawned to modify the heading and body text. Concurrently on main, conflicting text is committed on the exact same lines. Attempting git merge feature triggers Git's conflict detection, producing conflict markers in the file. The engineer manually evaluates both inputs, constructs a synthesized version combining both feature and main requirements, stages index.html, commits the merge resolution, and pushes both branches to GitHub.

### 6. Implementation / Procedure

**Step 1:** Initialized local Git repository with default branch: git init -b main and configured identity: git config user.name "Raghava m".

**Step 2:** Created initial baseline index.html and committed: git add index.html; git commit -m "Initial commit with index page" (Commit: aee5835).

**Step 3:** Created and switched to feature branch: git checkout -b feature.

**Step 4:** Modified index.html on feature branch to "Welcome to Feature Branch" and committed (Commit: 452eb12).

**Step 5:** Switched back to main branch and made conflicting change on the same lines to "Welcome to Main Website" (Commit: a1a2f4b).

**Step 6:** Executed merge: git merge feature, triggering CONFLICT (content): Merge conflict in index.html.

**Step 7:** Inspected conflict markers in index.html demarcated by <<<<<<< HEAD, =====, and >>>>>>> feature.

**Step 8:** Manually edited index.html to combine updates into "Welcome to Main Website - Feature

Update".

**Step 9:** Staged resolved file and completed merge commit: `git commit -m "Resolve merge conflict between main and feature"` (Commit: 971f733).

**Step 10:** Verified commit DAG history tree: `git log --oneline --graph --all`.

**Step 11:** Connected remote repository: `git remote add origin https://github.com/Raghavendra-2007/Git-Branching-and-Merge-Conflict-Resolution.git`.

**Step 12:** Pushed both branches: `git push -u origin main` and `git push -u origin feature`.

---

RAJKOT-MORBI ROAD, AT & PO: GAURIDAD, RAJKOT - 360 003, GUJARAT, INDIA

Ph: +91-281-2923112, +91-281-2924154-56 | email: [info@marwadiuniversity.ac.in](mailto:info@marwadiuniversity.ac.in) | [www.marwadiuniversity.ac.in](http://www.marwadiuniversity.ac.in)

## 7. Testing and Results

| Test / Parameter               | Expected Result                                    | Obtained Result                                         |
|--------------------------------|----------------------------------------------------|---------------------------------------------------------|
| <b>Branch Isolation</b>        | Feature branch created independently of main       | Branch created; independent HEAD pointers maintained    |
| <b>Divergent Modifications</b> | Commits made to identical lines in both branches   | Commit a1a2f4b (main) and 452eb12 (feature) created     |
| <b>Conflict Detection</b>      | Git halts auto-merge on conflicting lines          | CONFLICT (content): Merge conflict in index.html        |
| <b>Conflict Markers</b>        | Markers indicate HEAD vs feature text differences  | Markers <<<<<<<, =====, >>>>>>> accurately embedded     |
| <b>Manual Resolution</b>       | Clean file created with markers removed and staged | Resolved file committed with merge hash 971f733         |
| <b>Commit History Graph</b>    | Graphical DAG reflects converging branch paths     | Tree verified via git log --oneline --graph --all       |
| <b>Remote Synchronization</b>  | Both branches synchronized to GitHub remote        | Verified on GitHub: origin/main and origin/feature live |

## 8. Challenges Faced and Solutions

| Challenge                                                          | Solution                                                                                                        |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Automatic 3-way merge failure due to overlapping line edits</b> | Analyzed conflict markers and manually reconciled both branch features into a unified layout without data loss. |
| <b>Risk of corrupting working tree during conflict state</b>       | Inspected git status at each transition and verified staging area before finalizing the merge commit.           |
| <b>Multi-branch remote synchronization to GitHub</b>               | Configured upstream tracking for both main and feature using git push -u origin <branch>.                       |

## 9. Conclusion

The complex engineering problem of managing concurrent branch modifications and reconciling merge conflicts in distributed version control was successfully solved. By simulating realistic divergent changes across 'main' and 'feature' branches, the automated failure mechanism was analyzed and corrected through manual conflict resolution. The outcome is a verified, fully merged codebase with preserved commit history published to GitHub (<https://github.com/Raghavendra-2007/Git-Branching-and-Merge-Conflict-Resolution>). This practical established robust competence in collaborative Git workflows and conflict mitigation.

## 10. SDG Mapping and Justification

**SDG 4 (Quality Education):** Promotes mastery of modern, industry-standard collaborative engineering practices and version control systems critical for professional software careers.

**SDG 9 (Industry, Innovation, and Infrastructure):** Fosters reliable, collaborative software infrastructure through open-source version control and distributed development practices.

## 11. References (in IEEE Style)

- [1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. New York, NY, USA: Apress, 2014.
- [2] J. Loeliger and M. McCullough, *Version Control with Git: Powerful tools and techniques for collaborative software development*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2012.
- [3] Git Documentation, "git-merge - Join two or more development histories together," *Git SCM Documentation*, 2026. [Online]. Available: <https://git-scm.com/docs/git-merge>.